



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



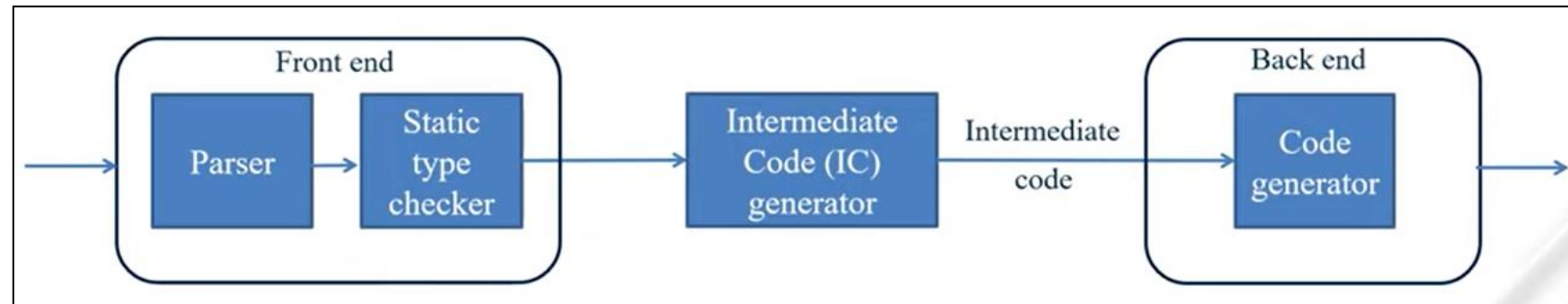
MODULE
4

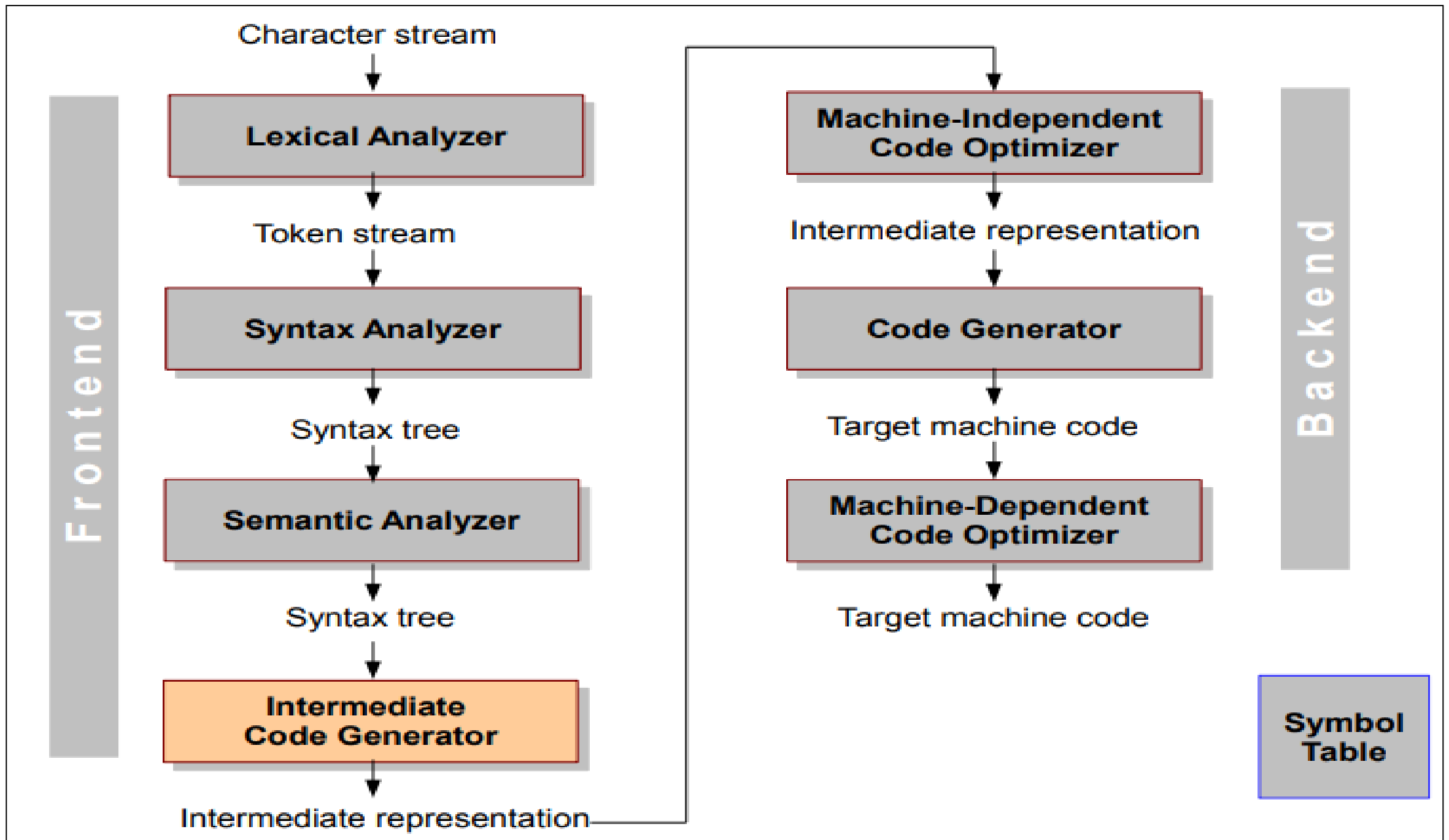
INTERMEDIATE CODE GENERATION

Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

INTERMEDIATE CODE GENERATION

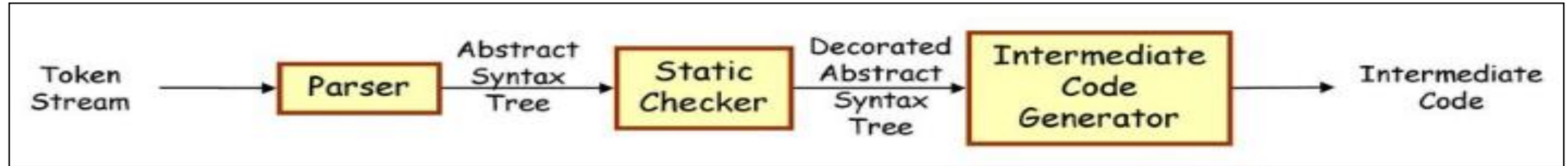
- Intermediate code can translate the source program into the machine program.
- Intermediate code is generated because the compiler can't generate machine code directly in one pass.
- First, it converts the source program into intermediate code, which performs efficient generation of machine code further.
- Intermediate code generator receives input from its predecessor phase and semantic analyzer phase. It **takes input in the form of an annotated syntax tree**.
- The intermediate code can be represented in the form of
 - **Postfix notation**
 - **Syntax tree**
 - **Directed acyclic graph**
 - **Three address codes**
 - **Quadruples**
 - **Triples**





Static Checker

- Static checking confirms that the compiler can *compile the program successfully*.
- It identifies the *programming errors* earlier.



- Static checking performs two types of checking:

(i) Syntactic Checking

- This kind of checking *identifies the syntactic errors* present in the program.
- The compiler examines the sequence of elements of the program.

(ii) Type Checking

- Type checking is the process of *verifying and enforcing constraints of types* in values.
- The type-checker determines whether these values are used appropriately or not.

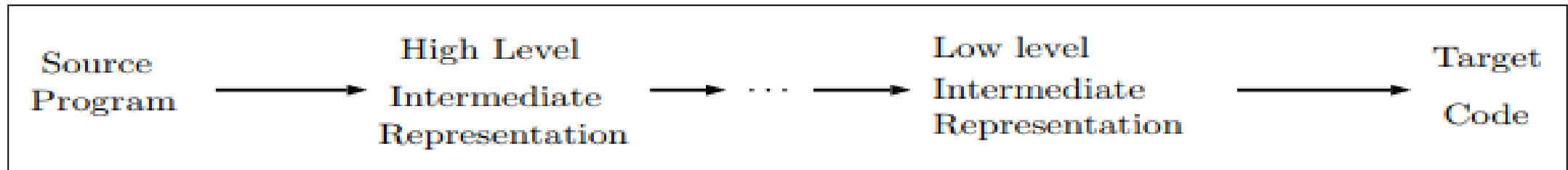
An error is reported if such an enclosing statement does not exist.

Example

Static checking assures that a **break-statement** in C is enclosed within a **while**, **for**, or **switch** statement;

Intermediate Representation

- Intermediate codes can be represented in a variety of ways and they have their own benefits.
- The intermediate representation must be:
 - *Easy to produce.*
 - *Easy to translate into target code.*



1. High Level Intermediate Code

- This representation is the very first intermediate representation of the *source language*.
- The **syntax tree** generated by the parser is the high-level representation.
- The high-level intermediate representation provides a **hierarchical structure of the source program**.
- This hierarchical structure helps the task such as a static check.

2. Low Level Intermediate Code

- The intermediate code generation phase generates the low-level intermediate representation.
- The **three-address code** is a low-level representation.
- The three-address code is suitable for machine-dependent tasks.
- Suitable for **register** and **memory allocation**

*Intermediate code can be either **language specific** (e.g., Byte Code for Java) or **language independent** (three-address code).*

Variants of Syntax Trees

- Nodes in a syntax tree represent constructs in the source program
- Children of a node represent the meaningful components of a construct.

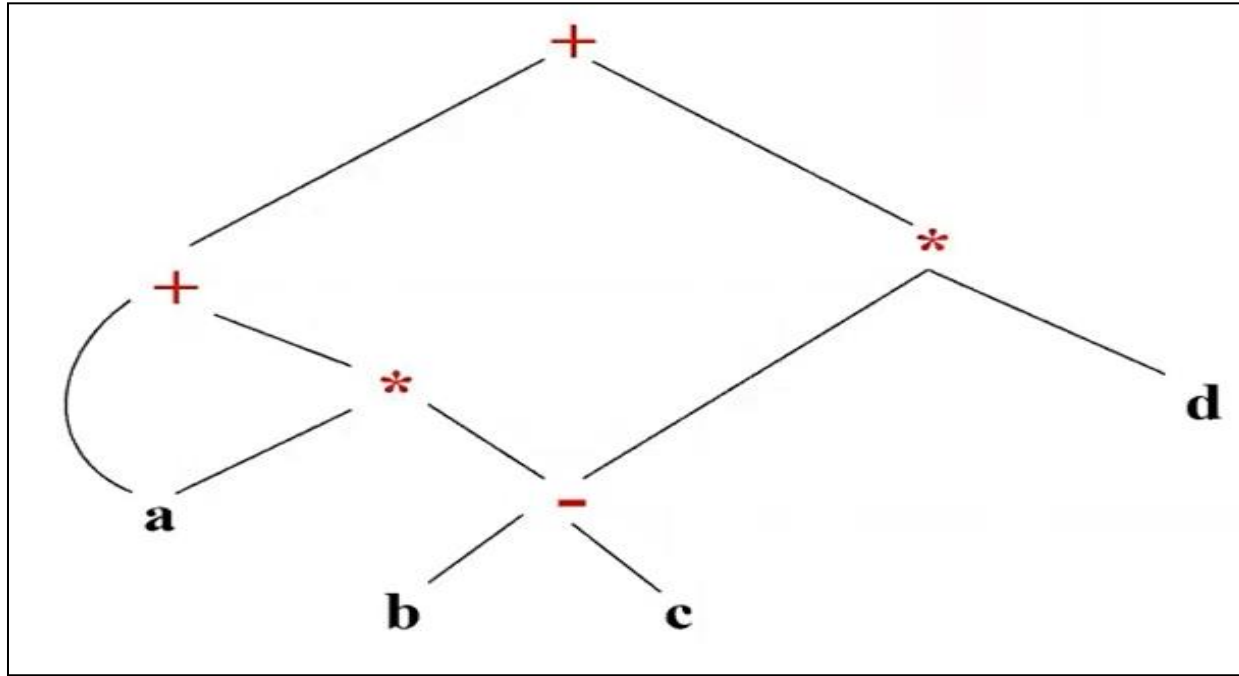
Directed Acyclic Graphs (DAG) for Expressions

- A directed acyclic graph for an expression *identifies* the **common sub expressions** (sub expressions that occur more than once) of the expression.
- DAG's can be constructed by using the same techniques that construct syntax trees.
- A DAG has
 - **Leaves** corresponding to atomic **operands** (identifiers, names, or constants)
 - **Interior nodes** corresponding to **operators**.

- A node N in a **DAG** has more than one parent if N represents a common sub expression.

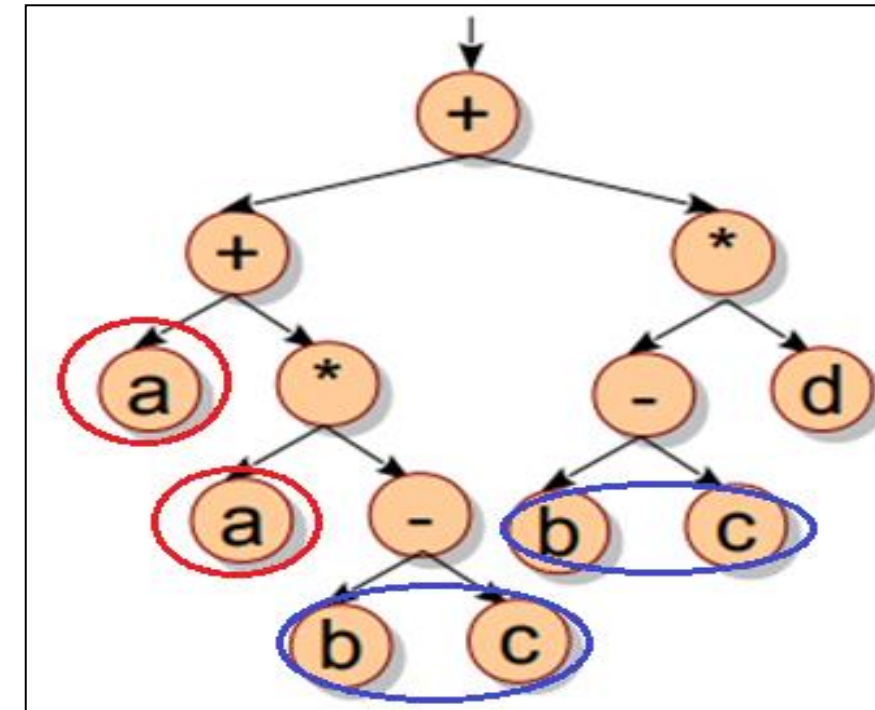
- In a **syntax tree**, the tree for the common subexpression would be replicated as many times as the subexpression appears in the original expression

Example 1 : Construct DAG for the expression



$a + a * (b - c) + (b - c) * d$

Syntax Tree



- **Operators** : +, *, - (Internal Nodes)
- **Operands** : a,b,c,d (Leaf)
- Leaf for “**a**” has two parents (+,*), because “**a**” appears twice in the expression.
- The two occurrences of the common **subexpression** “**b - c**” are represented by one node, the node labeled “**—**”.
- That node has two parents, representing its two uses in the subexpressions **a * (b - c)** and **(b-c)*d**

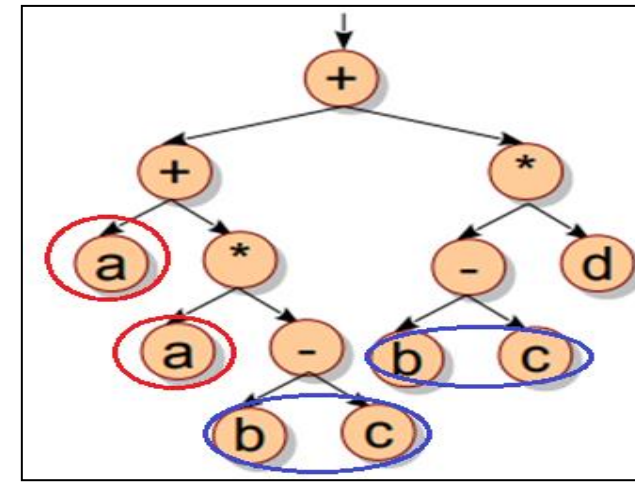
Example 2: Syntax Directed Definition (SDD) to produce syntax trees or DAG

- The functions **Leaf** and **Node** created a fresh node each time they were called.
- It will construct a DAG if, before creating a new node, these functions first check whether an identical node already exists.
 - If a previously created identical node exists, the existing node is returned.

Productions	Semantic Rules
$E \rightarrow E_1 + T$	$E.node = \text{new Node}(\text{'+'}, E_1.node, T.node)$
$E \rightarrow E_1 - T$	$E.node = \text{new Node}(\text{'-'}, E_1.node, T.node)$
$E \rightarrow T$	$E.node = T.node$
$T \rightarrow (E)$	$T.node = E.node$
$T \rightarrow \text{id}$	$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$
$T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$

- For instance, before constructing a new node, **Node(op, left, right)** we check whether there is already a node with label **op**, and children **left** and **right**, in that order. If so, **Node** returns the existing node; otherwise, it creates a new node.

- Assume that **entry-a** points to the **symbol-table entry for a**, and similarly for the other identifiers.



```

p1 = Leaf(id, entry-a)
p2 = Leaf(id, entry-a) = p1
p3 = Leaf(id, entry-b)
p4 = Leaf(id, entry-c)
p5 = Node('-', p3, p4)
p6 = Node('*', p1, p5)
p7 = Node('+', p1, p6)
p8 = Leaf(id, entry-b) = p3
p9 = Leaf(id, entry-c) = p4
p10 = Node('-', p3, p4) = p5
p11 = Leaf(id, entry-d)
p12 = Node('*', p5, p11)
p13 = Node('+', p7, p12)

```

- Leaf(id, entry-a)** is repeated at **step 2**, the node created by the previous call is returned.

p2 = p1

- The **nodes** returned at **steps 8 and 9** are the same as those returned at **steps 3 and 4**.

p8 = p3
p9 = p4

- The node returned at **step 10** must be the same as that returned at **step 5**.

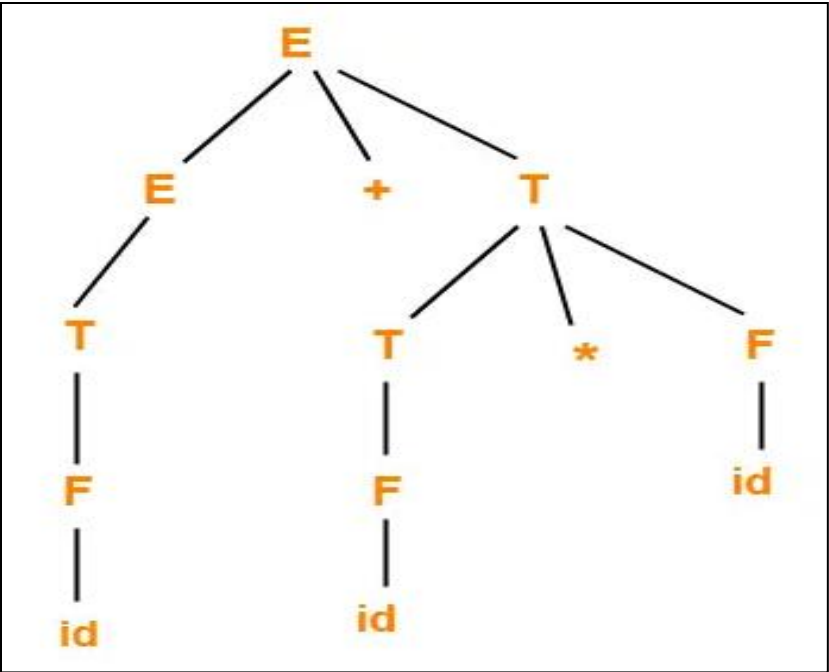
p10 = p5

Construct parse tree, syntax tree and the DAG for the given grammar for the input string

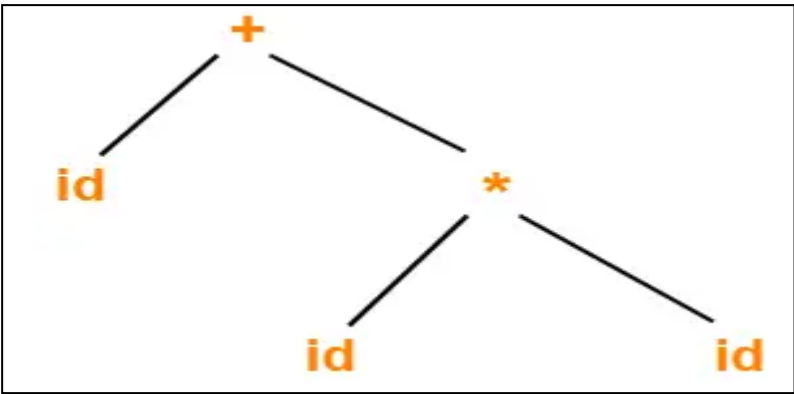
*id + id * id*

$E \rightarrow E + T \mid T$
 $T \rightarrow T \times F \mid F$
 $F \rightarrow (E) \mid id$

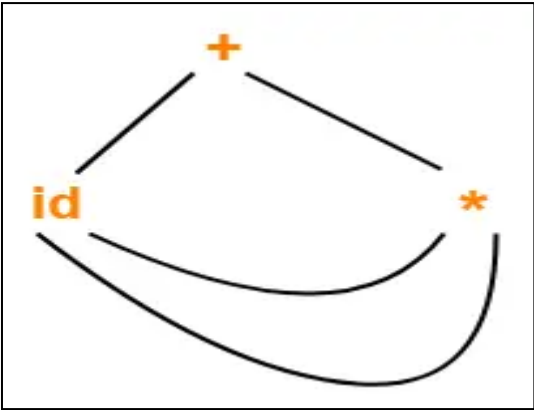
Parse Tree



Syntax tree



DAG

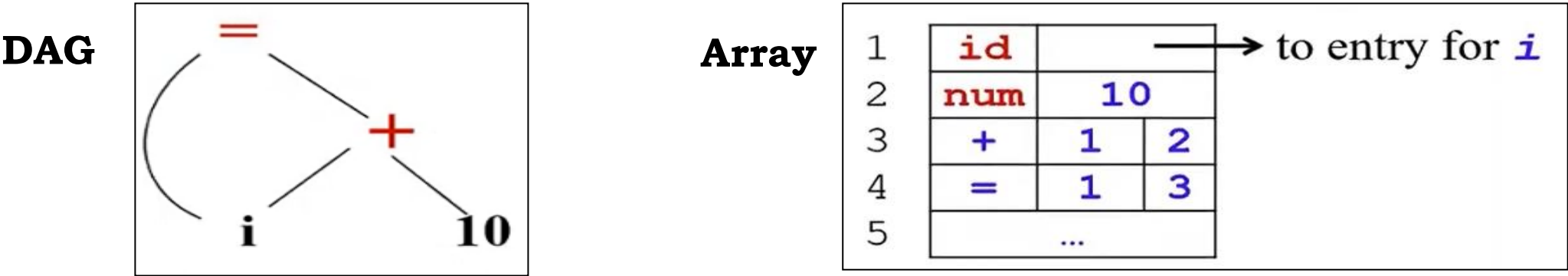


Value-Number Method for Constructing DAG's

- Nodes of a syntax tree or DAG are stored in an *array of records*.
- The **integer index** of the record for a node in the array is known as the **value number** of that node.
- Each row of the array represents one record, and therefore one node.
- In each record, the **first field** is an **operation code**, indicating the **label of the node**.

Leaves	One additional field	Holds the lexical value (either a symbol-table pointer or a constant)
Interior nodes	Two additional fields	Indicating the left and right children

- **Example: Nodes of a DAG for $i = i + 10$ allocated in an array**



- The node labeled “+” has value number **3**, and its **left** and **right children** have value numbers **1** and **2**

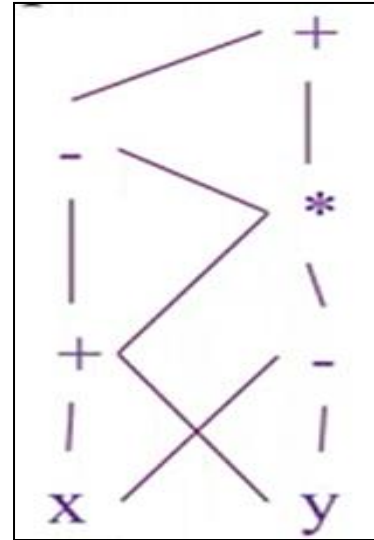
Algorithm : The value-number method for constructing the nodes of a DAG

INPUT	Label op , node l , and node r
OUTPUT	The value number of a node in the array with signature (op, l,r) .
METHOD	Search the array for a node M with label op, left child l, and right child r If there is such a node, return the value number of M If not, create in the array a new node N with label op, left child l, and right child r, and return its value number.

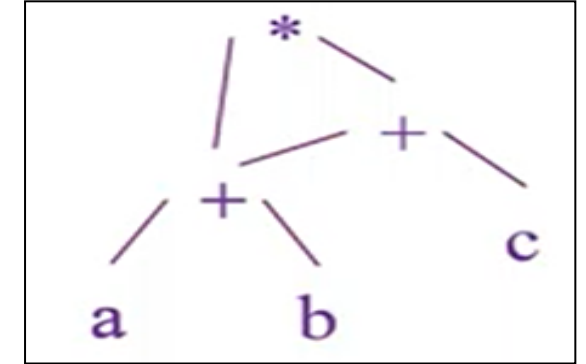
Construct the DAG for the expression

$$((\mathbf{x}+\mathbf{y})-((\mathbf{x}+\mathbf{y})^*(\mathbf{x}-\mathbf{y}))) + ((\mathbf{x}+\mathbf{y})^*(\mathbf{x}-\mathbf{y}))$$

Construct the DAG and identify the value numbers for the subexpressions of the following expressions, assuming + associates from the left.

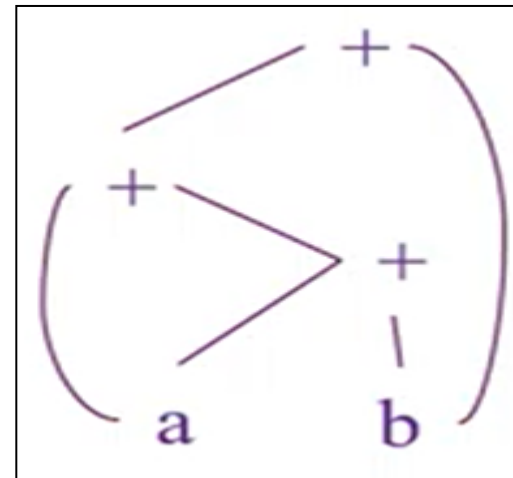


$$(a+b)*(a+b+c)$$

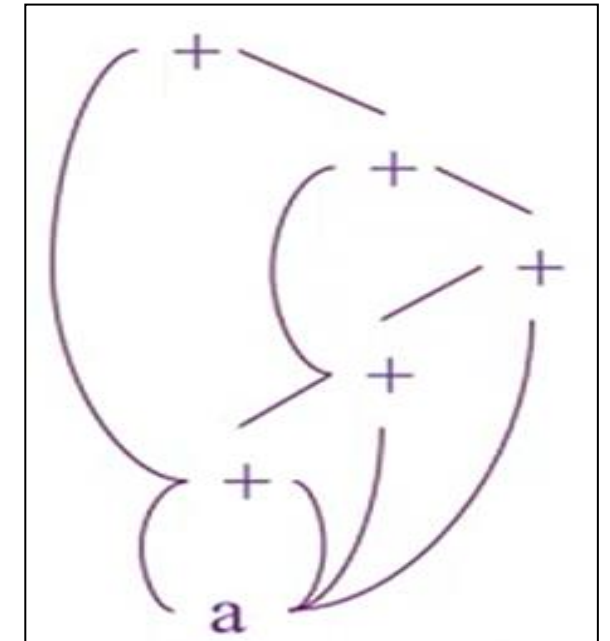
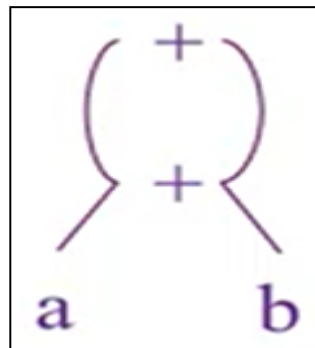


$$a + a + ((a + a + a + (a + a + a + a)))$$

a + b + a + b



$$a + b + (a + b)$$



Three-Address Code

- **Three address code** is a type of intermediate code which is easy to generate and can be easily converted to machine code.
- The three-address code is a sequence of statements of the form

$$a = b \text{ op } c$$

- **a**, **b**, **c** are **operands** (*programmer-defined names*, *constants*, or *compiler-generated temporary names*)
- **op** represents an **operator** (*constant* or *floating point arithmetic operators* or a *Boolean valued data* or a *logical operator*)
- The reason for the name “three address code” is that each statement generally includes **three addresses**, **two for the operands**, and **one for the result**.

General form of Three-Address Code Representation is



Steps to Convert an Expression to Three-Address Code

- **Identify Subexpressions:** Look for operations in the expression and identify subexpressions that can be computed separately.
- **Introduce Temporaries:** Use temporary variables to store intermediate results of these subexpressions.
- **Write Instructions:** Create a sequence of three-address instructions, using the temporary variables as needed.
- **Order of Operations:** Follow the precedence and associativity rules of the expression to ensure correct evaluation.

$x = (a + b) * (c - d) + e$

$t1 = a + b$
 $t2 = c - d$
 $t3 = t1 * t2$
 $x = t3 + e$

- The expression has two main parts:
 $a + b$ and $(c - d)$.
- The final operation is $*$ with these two results, followed by $+ e$.

$z = (x + y) * (a - b) / (c + d)$

$t1 = x + y$
 $t2 = a - b$
 $t3 = c + d$
 $t4 = t1 * t2$
 $z = t4 / t3$

$x = (a + b) * (c - d) + e / f$

$t1 = a + b$
 $t2 = c - d$
 $t4 = t1 * t2$
 $t3 = e / f$
 $x = t4 + t3$

$$\mathbf{x = (a + b) * (c - d) / e + f}$$

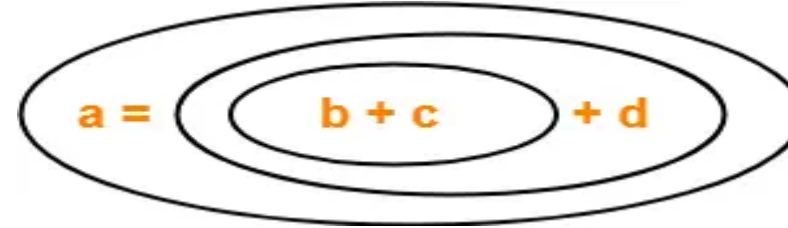
t1 = a + b
t2 = c - d
t3 = t1 * t2
t4 = t3 / e
x = t4 + f

Always evaluate expressions within parentheses first, treating them as higher precedence.

Write down the three address code for the given expression

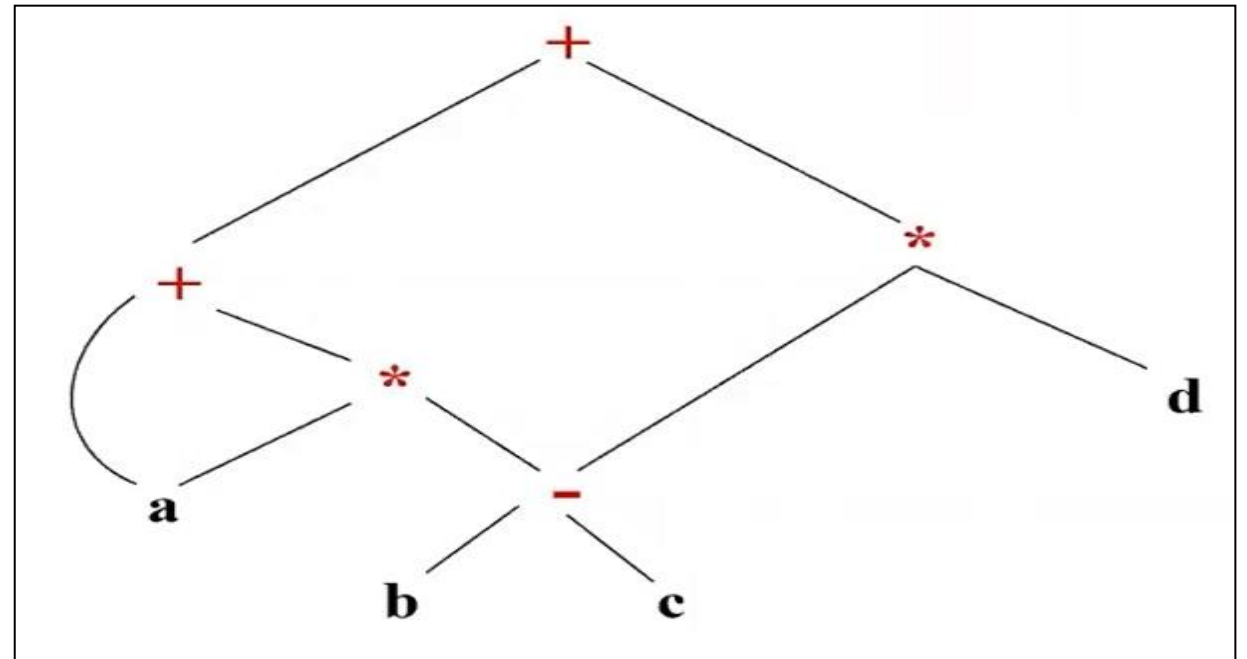
1. $a = (b+c) / d$

**$t_1 = b+c$
 $t_2 = t_1 / d$
 $a = t_2$**



2. $a + a * (b - c) + (b - c) * d$

**$t_1 = b - c$
 $t_2 = a * t_1$
 $t_3 = a + t_2$
 $t_4 = t_1 * d$
 $t_5 = t_3 + t_4$**



Addresses and Instructions

- Three-address code is built from two concepts:
 - **Addresses**
 - **Instructions**
- An **address** can be one of the following:

Address	Description
A name	In actual implementation, a source name is replaced by a pointer to its symbol-table entry , where all information about the name is kept.
A constant	A compiler must deal with many different types of constants and variables
A compiler-generated temporary	In optimizing compilers, to create a distinct name each time a temporary is needed. Example : a = (b+c) / d t1 = b+c t2 = t1 / d a = t2

Consider the following C-like code snippet:

```
int x = 10;  
float y = 20.5;  
x = x + 5;
```

Symbol Table

Name	Type	Memory Location	Scope
x	int	0x1001	Global
y	float	0x1002	Global

- When processing the assignment **x = x + 5**, the compiler will:
 - Replace the occurrence of x with a pointer or reference to its symbol table entry.
 - The actual value (10 for x) will be fetched from the memory location (0x1001) during execution.

- Common **three address instruction forms** (Types of three address statements)

Instruction	Form	Description
Assignment Instructions	$x = y \text{ op } z$	<u>op</u> is a binary arithmetic or logical operation, and <u>x</u>, <u>y</u>, and <u>z</u> are addresses. Example: $a = b + c * d$ $t1 = c * d$ $t2 = b + t1$ $a = t2$
	$x = \text{op } y$	<u>op</u> is a unary operation (unary minus, logical negation, shift operators, and conversion operators) Example: 1. Convert an integer to a floating-point number 2. $a = b * -c$ $t1 = -c$ $t2 = b * t1$ $a = t2$
Copy Instructions	$x = y$	<u>x</u> is assigned the value of <u>y</u>
Unconditional Jump	goto L	Three-address instruction with label <u>L</u> is the next to be executed .

Instruction	Form	Description
Conditional Jumps	if x goto L if false x goto L	Execute the instruction with label L next, if x is true and when x is false respectively.
	if x <i>relop</i> y goto L	Apply a relational operator (<, ==, >=, etc.) to x and y, and execute the instruction with label L next if x stands in relation relop to y.
Procedure and Functions Calls, and returns	param x call p,n return y	p is a function which takes x as a parameter and returns y. For a procedure call <i>p(x1, ..., xn)</i> param x1 ... param xn call p, n // n represent number of parameters y, representing a returned value (optional)
Indexed copy Instructions	x = y[i] and x[i] = y	x = y[i] , value of i th position of array y is assigned to x. x[i] = y , value of y is assigned to i th position of array x.
Address and Pointer Assignments	x = &y x = *y *x = y	x = & y , the address of y is assigned to x. x = * y , Content of location pointed to by y is assigned to x. *x = y , sets the r-value of the object pointed to by x to the r-value of y.

- **Symbolic labels** will be used by instructions that **alter the flow of control**.
- A symbolic label represents the index of a three-address instruction in the sequence of instructions.

Example : Find out the translation of the following statement:

do $i = i + 1$;
while ($a[i] < v$);

Translation with **symbolic labels**

L : $t1 = i + 1$
 $i = t1$
 $t2 = i * 4$
 $t3 = a[t2]$
 if $t3 < v$ goto **L**

Translation with **position number**

100 $t1 = i + 1$
101 $i = t1$
102 $t2 = i * 4$
103 $t3 = a[t2]$
104 if $t3 < v$ goto **100**

- **Multiplier 4** is used as each memory location of integer is considered as of 4-byte integer data.
- **v** represent **value**

Implementation of Three Address Code

- There are 3 representations of three address code namely

Quadruple

Triples

Indirect Triples

1. Quadruple

- The quadruples have four fields to implement the three address code.

Field	Description	Example
op	Used for storing the internal code of the operator	+
arg1	Used for storing the operands	y
arg2	Used for storing the operands	x
result	Used for storing the result of the expression	z

Example : Consider three-address instruction **$z = y + x$**

Exceptions to this rule

- Instructions with **unary operators** do not use **arg2**
- param** neither uses **arg2** nor **result**
- Conditional** and **unconditional jumps** put the target label in **result**.

$x = -y$ or $x = y$

Example :

Consider expression $x = (a + b) * (c - d)$

The **three address code** is:

```
t1 = a + b  
t2 = c - d  
x  = t1 * t2
```

Quadruples

op	arg 1	arg 2	result
+	a	b	t1
-	c	d	t2
*	t1	t2	x

2. Triples

- A triple three-address implementation will have **three fields**.

Field	Description
op	Used for storing the internal code of the operator
arg1	Used for storing the operands
arg2	Used for storing the operands

- The result field in quadruples is used primarily for temporary names.
- *The result is not explicitly stored in a separate field.*
- In triples, we refer to the **result of an operation x op y by its position**, rather than by an explicit temporary name.
- Instead of the temporary t_i in quadruples, **a triple representation would refer to position (0)**.
- Parenthesized numbers represent pointers into the triple structure itself.
- Triple is equivalent to DAG while representing expressions.

Example :

Consider expression $x = (a + b) * (c - d)$

The **three address code** is:

```
t1 = a + b
t2 = c - d
x = t1 * t2
```

Triples

Index
(0)
(1)
(2)
(3)

op	arg 1	arg 2
+	a	b
-	c	d
*	(0)	(1)
=	x	(2)

t1 position (0)

t2 position (1)

t3 position (2)

t4 position (3)

3. Indirect Triples

- This representation is an enhancement over triples representation.
- It uses an additional **instruction array to list the pointers** to the triples in the desired order.
- Thus, instead of position, **pointers are used to store the results**.
- It allows the optimizers to easily reposition the sub-expression for producing the optimized code

Example :

Consider expression $x = (a + b) * (c - d)$

The **three address code** is:

t1	=	a	+	b
t2	=	c	-	d
x	=	t1	*	t2

Index	Pointer Array
(0)	(11)
(1)	(12)
(2)	(13)
(3)	(14)

Index	op	arg 1	arg 2
(0)	+	a	b
(1)	-	c	d
(2)	*	(11)	(12)
(3)	=	x	(13)

Example :

Consider expression $a = b * -c + b * -c$

The **three address code** is:

t1 = minus **c** or **-c**
t2 = **b** * **t1**
t3 = minus **c** or **-c**
t4 = **b** * **t3**
t5 = **t2** + **t4**
a = **t5**

The special operator minus is used to distinguish the unary minus operator as in -c from the binary minus operator as in b-c.

Quadruples

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>	<i>result</i>
0	-	c		t ₁
1	*	b	t ₁	t ₂
2	-	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a

Expression $a = b * -c + b * -c$

$t1 = \text{minus } c \text{ or } -c$
 $t2 = b * t1$
 $t3 = \text{minus } c \text{ or } -c$
 $t4 = b * t3$
 $t5 = t2 + t4$
 $a = t5$

Triple

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>
0	-	c	
1	*	b	(0)
2	-	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

t1 position (0)
t2 position (1)
t3 position (2)
t4 position (3)
t5 position (4)

Indirect Triple

	<i>instruction</i>
30	(0)
31	(1)
32	(2)
33	(3)
34	(4)
35	(5)

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>
0	-	c	
1	*	b	(0)
2	-	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

- Write quadruple, triples and indirect triples for following expression :

$$(x + y) * (y + z) + (x + y + z)$$

The three address code is:

t1 = x + y
t2 = y + z
t3 = t1 * t2
t4 = t1 + z
t5 = t3 + t4

#	Op	Arg1	Arg2	Result
(1)	+	x	y	t1
(2)	+	y	z	t2
(3)	*	t1	t2	t3
(4)	+	t1	z	t4
(5)	+	t3	t4	t5

#	Op	Arg1	Arg2
(1)	+	x	y
(2)	+	y	z
(3)	*	(1)	(2)
(4)	+	(1)	z
(5)	+	(3)	(4)

#	Op	Arg1	Arg2
(14)	+	x	y
(15)	+	y	z
(16)	*	(14)	(15)
(17)	+	(14)	z
(18)	+	(16)	(17)

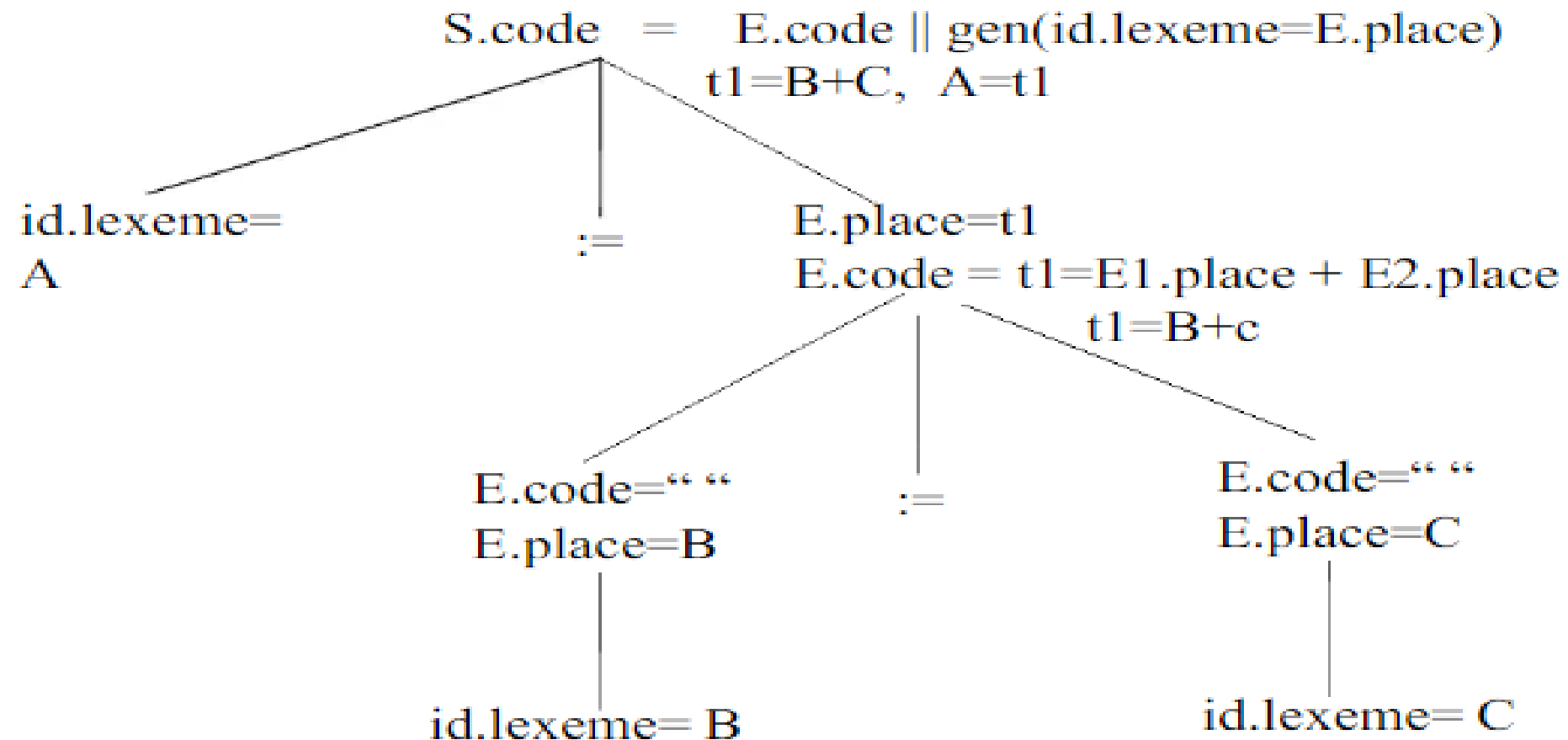
List of pointers to table

#	Statement
(1)	(14)
(2)	(15)
(3)	(16)
(4)	(17)
(5)	(18)

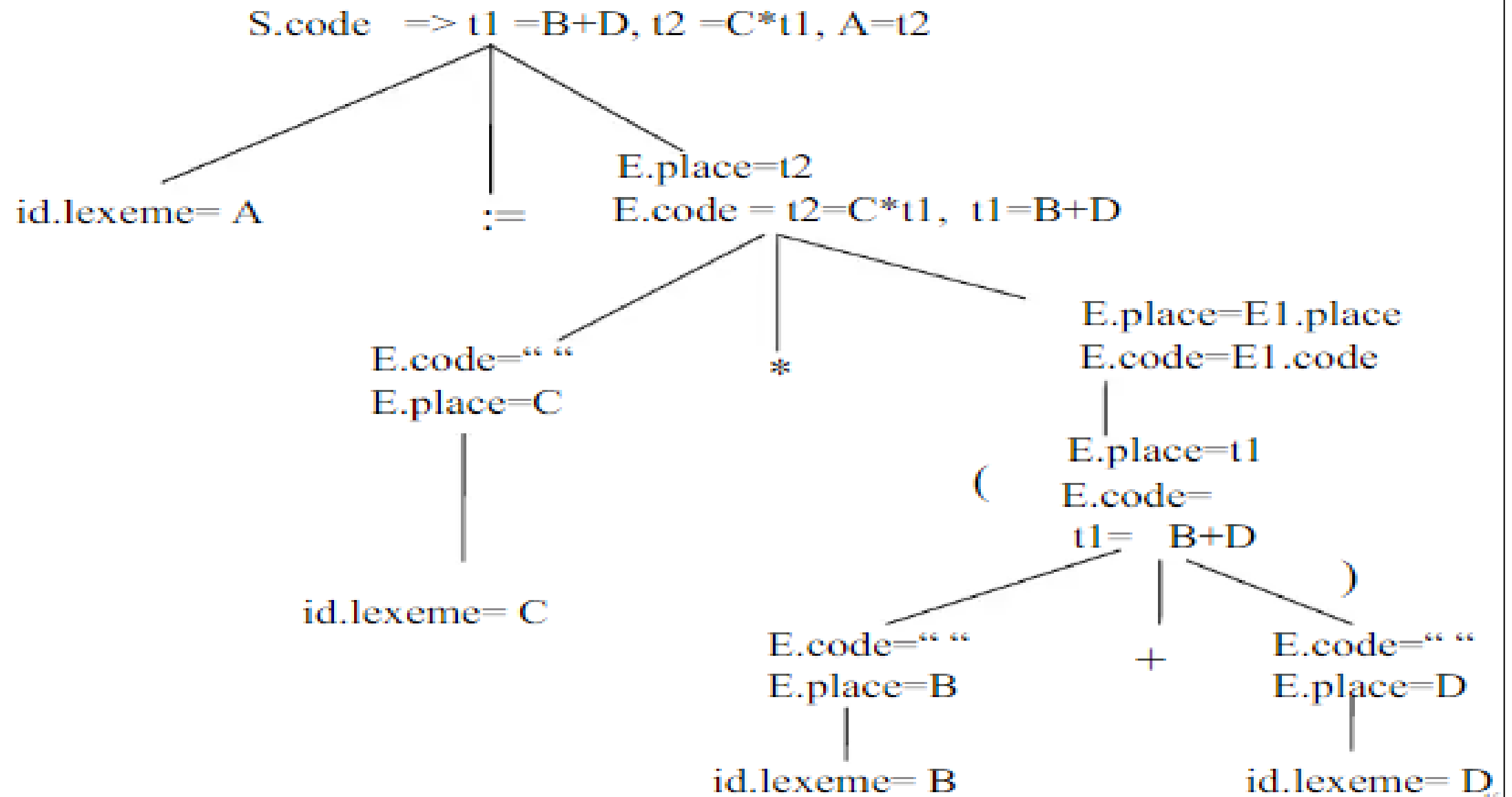
Indirect Triples representation

Examples

Three address code for $A := B + C$



Three address code for $A := C * (B + D)$



**while a < b
a = (a + b) * c**



**L1: t1 := a < b
if t1 = 0 goto L2
t2 := a + b
t3 := t2 * c
a := t3
goto L1
L2:**

**i := 2 * n + k
while i do
i := i - k**



**t1 := 2
t2 := t1 * n
t3 := t2 + k
i := t3
L1: if i = 0 goto L2
t4 := i - k
i := t4
goto L1
L2:**

How come? Draw the decorated version

1. Generate the three-address code for the following program segment

```
while(x < z and y > s) do
  if x = 1 then
    z = z + 1
  else
    while x <= s do
      x = x + 10;
```

The three-address code for the given program segment is given below:

```
1. if x < z goto (3)
2. goto (16)
3. if y > s goto (5)
4. goto (16)
5. if x = 1 goto (7)
6. goto (10)
7. t1: = z + 1
8. z: = t1
9. goto (1)
10. if x <= s goto (12)
11. goto (1)
12. t2: = x + 10
13. x: = t2
14. goto (10)
15. goto (1)
16. Next
```

2. Generate the three-address code for the following program segment

```
for (k = 1; k <= 12; k++)  
  if x < y then  
    a = b + c;
```

The three-address code for the given program segment is given below:

```
1. k: = 1  
2. if k <= 12 goto (4)  
3. goto (11)  
4. if x < y goto (6)  
5. goto (8)  
6. t1: = b + c  
7. a: = t1  
8. t2: = k + 1  
9. k: = t2  
10. goto (2)  
11. Next
```

3. Generate the three-address code for the following program segment where, x and y are arrays of size 10 * 10, and there are 4 bytes/word.

The three-address code for the given program segment is given below:

```
begin
add = 0
a = 1
b = 1
do
begin
add = add + x[a,b] * y[a,b]
a = a + 1
b = b + 1
end
while a <= 10 and b <= 10
end
```

```
1. add: = 0
2. a: = 1
3. b: = 1
4. t1: = a * 10
5. t1: = t1 + b
6. t1: = t1 * 4
7. t2: = addr(x) - 44
8. t3: = t2 [t1]
9. t4: = b * 10
10. t4: = t4 + a
11. t4: = t4 * 4
```

```
12. t5: = addr(y) - 44
13. t6: = t5[t4]
14. t7: = t3 * t6
15. t7: = add + t7
16. t8: = a + 1
17. a: = t8
18. t9: = b + 1
19. b: = t9
20. if a <= 10 goto (22)
21. goto (23)
22. if b <= 10 goto(4)
23. Next
```

4. Generate three address code for the following code

```
switch (ch)
{
case 1 : c = a + b;
break;
case 2 : c = a - b;
break;
}
```

The three-address code for the given program segment is given below:

```
if ch = 1 goto L1
if ch = 2 goto L2
L1:
    t1 = a + b
    c = t1
    goto Next
L2:  t2 = a - b
    c = t2
    goto Next
Next:
```

5. Translate the following program segment into three-address statements:

```
switch(a + b)
{
case 2: {x = y;
break;}
case 5: switch x
{
case 0: {a = b + 1;
break;}
}
case 1: {a = b + 3; break;}
default: {a = 2; break;}
}
break;
case 9: {x = y - 1; break;}
default: {a = 2; break;}
}
```

The three-address code for the given program segment is given below:

```
1. t1 := a + b
2. goto (23)
3. x := y
4. goto (27)
5. goto (14)
6. t3 := b + 1
7. a := t3
8. goto (27)
9. t4 := b + 3
10. a := t4
11. goto (27)
12. a := 2
13. goto (27)
```

```
14. if x = 0 goto (6)
15. if x = 1 goto (9)
16. goto (12)
17. goto (27)
18. t5 := y - 1
19. a := t5
20. goto (27)
21. a := 2
22. goto (27)
23. if t = 2 goto (3)
24. if t = 5 goto (5)
25. if t = 9 goto (18)
26. goto (21)
27. Next
```

6. Generate three address code for the following code

```
c = 0
do
{
if (a < b) then
x++;
else
x--;
c++;
} while (c < 5)
```

The three-address code for the given program segment is given below:

```
1. c = 0
2. if (a < b) goto (4)
3. goto (7)
4. T1 = x + 1
5. x = T1
6. goto (9)
7. T2 = x - 1
8. x = T2
9. T3 = c + 1
10. c = T3
11. if (c < 5) goto (2)
```

7. Generate three address code for the following code

```
prod = 0 ;  
i = 1 ;  
do  
{  
  prod = prod + a[ i ] x b[ i ] ;  
  i = i + 1 ;  
} while (i <= 10) ;
```

The three-address code for the given program segment is given below:

```
1. prod = 0  
2. i = 1  
3. t1 = 4 x i  
4. t2 = a[t1]  
5. t3 = 4 x i  
6. t4 = b[t3]  
7. t5 = t2 x t4  
8. t6 = t5 + prod  
9. prod = T6  
10. t7 = i + 1  
11. i = t7  
12. if (i <= 10) goto (3)
```

Address & Pointers:

$x = \&y;$

$*a = b;$

$p = *q;$

Intermediate Code:

100: $t_1 = \&y$

101: $x = t_1$

102: $t_2 = *a$

103: $t_2 = b$

104: $t_3 = *q$

105: $p = t_3$

op	arg1	arg2	result
&	y	-	t_1
=	t_1	-	x
*	a	-	t_2
=	b	-	t_2
*	q	-	t_3
=	t_3	-	p

Quadruples

	op	arg1	arg2
0	&	y	-
1	=	x	(0)
2	*	a	-
3	=	(2)	b
4	*	q	-
5	=	p	(4)

Triples

Program Segment:

```
int fact(int n)
{ if(n == 0) return 1;
  else return (n * fact(n - 1));
}
```

Intermediate Code:

```
100:    function begin fact
101:    if (n == 0) goto 108
102:    t1 = n - 1
103:    param t1
104:    refparam res
105:    call fact, 2
106:    t3 = n * res
107:    return t3
108:    return 1
109:    function end
110:    exit
```

Program Segment:

```
do
{
    i = i + 1;
} while(a[i] < x);
```

Intermediate Code:

```
L:      t1 = i + 1
        i = t1
        t2 = i * 4
        t3 = a[t2]
        if t3 < x goto L
```

Intermediate Code:

```
100:    t1 = i + 1
101:    i = t1
102:    t2 = i * 4
103:    t3 = a[t2]
104:    if t3 < x goto 100
```

op	arg1	arg2	result
+	i	1	t ₁
=	t ₁	-	i
*	i	4	t ₂
= []	a	t ₂	t ₃
<	t ₃	x	t ₄
if	t ₄	L	-

Quadruples

	op	arg1	arg2
0	+	i	1
1	=	i	(0)
2	*	i	4
3	= []	a	(2)
4	<	(3)	x
5	if	(4)	L

Triples